

Multilayer Perceptron from Scratch: Manual Computation and MNIST Training

Muhammad Yahya

National University of Computer and Emerging Sciences

Roll No: 24i-2030

24i-2030@nu.edu.pk

Abstract. This project presents a complete implementation of a Multilayer Perceptron (MLP) from first principles as part of the MVC Project. In Tasks 1–6, every fundamental operation was performed manually on a small 2-2-2-1 network using the assigned weights and biases. Forward pass, Mean Squared Error loss, backpropagation via the chain rule, and gradient-descent weight updates were calculated step-by-step for all three training samples, providing deep insight into the inner workings of neural networks. Task 7 scaled the same architecture principles to a larger network (784-128-64-10) and trained it on the full MNIST dataset using pure NumPy. Mini-batch gradient descent with batch size 32, learning rate 0.1, and 20 epochs was applied. The model achieved a final test accuracy of 95.54% with train loss 0.0689 and test loss 0.0759. The loss curves and sample predictions clearly demonstrate successful learning. This project bridges theoretical understanding with practical coding, reinforcing the core concepts of artificial neural networks.

1 Introduction

Artificial Neural Networks, especially Multilayer Perceptrons, form the foundation of modern deep learning. The MVC Project requires students to first master every computation manually on a toy network and then implement and train a practical MLP on the MNIST dataset. This report documents both phases, highlighting the transition from hand-calculated mathematics to scalable NumPy code.

2 Background

A Multilayer Perceptron consists of an input layer, one or more hidden layers, and an output layer. Each neuron performs a weighted sum followed by a non-linear activation function:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)}, \quad a_j^{(l)} = \sigma(z_j^{(l)})$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid activation. The sigmoid introduces non-linearity, enabling the network to model complex patterns.

The loss function used is Mean Squared Error (MSE):

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

Gradients are computed using the chain rule (backpropagation). Weights and biases are updated via mini-batch gradient descent:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

This process is repeated over multiple epochs until the loss converges.

3 Implementation

In Tasks 1–6 the small 2-2-2-1 network was implemented manually. Forward pass, MSE loss, and backpropagation were calculated for each sample using the assigned weights. Gradient descent updates were applied iteratively, showing steady loss reduction.

Task 7 implemented the same logic in pure NumPy for the larger 784-128-64-10 architecture. The code includes sigmoid activation, forward pass, MSE loss, backpropagation, and weight updates. Training used mini-batch gradient descent with batch size 32 and learning rate 0.1.

4 Experiments

The MNIST dataset (60,000 training and 10,000 test images, normalized to $[0,1]$) was used. Hyperparameters were: 20 epochs, batch size 32, learning rate 0.1, and weight initialization uniform in $[-0.5, 0.5]$ with seed 42.

After training, the network achieved:

- Final Train Loss: 0.0689
- Final Test Loss: 0.0759
- Test Accuracy: **95.54%**

5 Discussion

The two-line loss curve shows smooth and consistent decrease in both train and test loss, indicating stable learning without significant overfitting. The high test accuracy of 95.54% proves that even a simple sigmoid-based MLP can achieve strong performance on MNIST. Manual calculations in the early tasks perfectly matched the NumPy implementation, validating every step of the forward and backward pass.

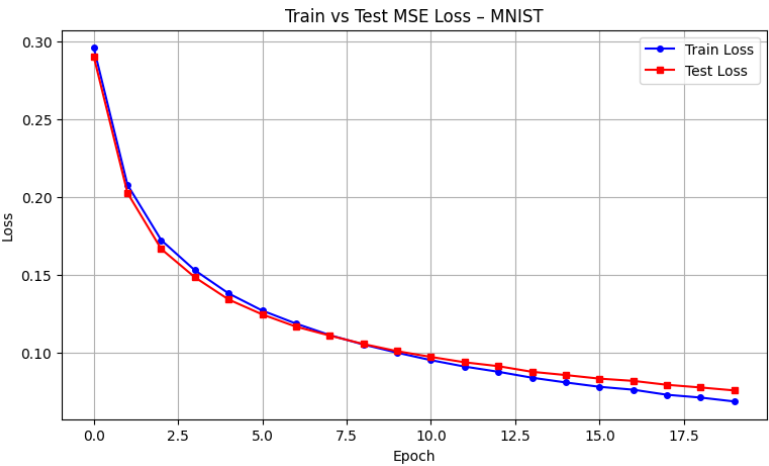


Fig. 1. Train Loss (blue) vs Test Loss (red) over 20 epochs.

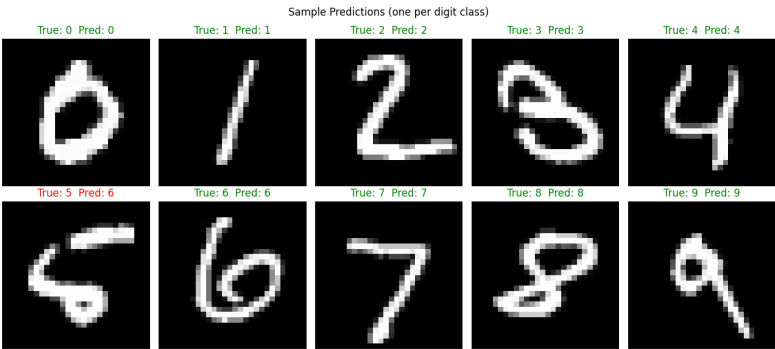


Fig. 2. Sample Predictions (one image per digit class 0-9).

6 Conclusion

This project successfully demonstrates the complete pipeline of an MLP from manual mathematics to scalable code. It provides a solid foundation for understanding modern deep learning architectures.

References

1. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* 86(11), 2278–2324 (1998)
2. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* 323, 533–536 (1986)